

北京市共享单车的投放量与运营方案的模型研究

——基于地铁站周边共享单车的调查

摘 要

本文主要解决北京市摩拜单车的分布和状态问题，从而给出北京市摩拜单车的投放数量和运营方式的优化方案。

【问题一】取北京市积水潭地铁站的经纬度所在的点作为中心原点，研究它周围的 $1\text{km} \times 1\text{km}$ 的正方形区域内的所有单车信息。每半个小时抓取一次研究区域内的摩拜单车的数据，并在地图上使其可视化，对其经纬度、数量等信息进行分析。以单位时间内整个区域的单车的总数量作为衡量单车分布在时间上不均衡的指标；同理，在同一时间段内，距离中心积水潭地铁站一定距离范围内的单车总量作为衡量单车在空间上分布不均衡的指标，得到单车在时间和空间上的分布状态和乘客使用规律。

【问题二】基于元胞自动机模型，利用积水潭地铁站周边的摩拜单车的投放量、租金等数据，考虑到活动优惠和车辆损毁因素，建立一个1行15列的矩阵，将各参数矩阵化，建立模拟出单车的分布和状态元胞自动机模型。在模型一的基础上，推导建立单车最大收益的函数模型，从而建立模型二，即模拟求解单车最佳投放数量的模型。结论是在距中心14L时，能够满足消费者的需求和获得最大的收益。因此在接下来的研究中，将针对这一种分布 $L=14$ 时进行研究。根据最佳投放方式，建立模型三，解决在最佳投放方式和分布方式情况下优惠活动对总收益的影响。结论是随着“烧钱活动”占单日总收益的比例增大，摩拜单车在两周内总的收益是减小的。

【问题三】针对北京市摩拜单车的分布和状态，我们提出了解决单车投放量和运行方式的优化方案。

一：最初投入以446辆单车的初始投入量为适宜投放量，且最好紧邻地铁站投入，目的是增加单车使用率，同时还能及时把报修车辆送修，有利于地铁站周边居民能更好使用单车。

二：基于问题二模拟的数周利润，“红包车”等优惠活动很难在短期内获利。建议减少这些活动的频率，以积累资本，在公司盈利后再进行扩展市场，回馈客户。

关键词： 摩拜单车;元胞自动机;网络爬虫;参数矩阵化;排队论

一 问题重述

1.1 问题背景

“最后一公里”[1]一直是公共交通出行方式未解决的问题之一，虽然之前有黑摩的，黑三轮的存在，但这些不仅增加了依赖公交的人群的出行的成本的同时也存在一定的社会隐患。在市政提出一些有桩自行车后虽然问题有所缓解，但是办理此类手续又相对繁琐，限制了急需车辆的人们。在此情况下的，“共享单车”的出现解决了原先有桩自行车的使用地区，手续的限制，利用手机APP的广泛应用，使得急需用车的人们可以方便用车，只要在城管规定的停车范围内停车即可，增加了人们低碳出行，有利于社会未来发展。

1.2 提出问题

为使乘客更方便的使用共享单车，使经营模式更有利于在短期内公司盈利，题目要求我们通过研究一些地点的车辆分布以及使用情况，来解决以下方面的问题：

- 问题一：以北京市某个小范围的数据为例，利用收集到的数据描述其共享单车分布与使用的规律。
- 问题二：考虑该品牌共享单车在北京市的运营方式，以及车辆折旧、损毁维护等因素，建立模型，解决如何投放才能做到既能满足消费者需求，又能尽早实现盈利，给出一个合理的单车投放数量和运营方式，并证明合理性。
- 问题三：要求我们以优化运营以及投放方式为目标导向，提出一些合理化建议，并撰写一封信给共享单车运营方。

1.3 问题要求的具体定义

研究的问题给出了定义不够清晰的两个要求：满足消费者需求；尽早实现盈利。下面给出这两个要求的具体阐述：

1.3.1 满足消费者需求

在一段时间内，在某区域的单车可利用数量始终大于等于同一时刻的该区域单车消费者的数量。

1.3.2 尽早实现盈利

对于尽早实现盈利，有如下两种理解：

- 不考虑共享单车的制造成本，在规定的完整时间段内，在某区域中单车依靠押金利息和租金收入获得的总体收入大于其损毁维护和活动优惠带来的总体支出，而且总体利润不小于同一条件下其他共享单车的平均总体利润。
- 考虑共享单车的制造成本，mobike品牌领先于其他同行进入盈利状态

二 问题分析

2.1 共享单车的品牌选择

我们将mobike单车作为研究对象。基对共享单车的领导品牌mobike和ofo进行比较，可以发

现：mobike单车只有两种车型，装备有简便的GPS定位仪；ofo单车具有多代性，管理较差。而且据mobike单车官方数据，已有30万共享单车投入北京，数量相对于其他少数品牌更具代表性。基于以上原因，我们将mobike单车作为研究对象。

2.2 调查地点的选择

我们选择了以积水潭地铁站为中心的 $1000 \times 1000(m^2)$ 范围，因为我们所调查的积水潭地铁站位于2号线，属于北京地铁的高客流量地段。如果可以设计出较为优良的方案，则可很广泛的使用于其他地区。 $1000 \times 1000(m^2)$ 的范围是因为，根据北京地铁站的数据显示，平均两站中的间隔距离约为1000m左右，所以为了考虑一个地铁站所造成的影响，我们以两站地的一半为调查范围。

2.3 数据收集

我们对 $1000 \times 1000(m^2)$ 的地区进行统计以积水潭为中心区域的共享单车的信息。通过使用经纬度定位进行网络爬虫收集数据，并同时进行APP截图及人工实地考察确认，最后使用收集的数据导入谷歌地图，定位我们所收集的数据点，使保证数据的准确性。

三 研究思路

本文为实现对北京市的共享单车运营方式的优化，通过使用网络爬虫与真实比对相结合的方式，调查积水潭地铁站周边共享单车的使用与分布情况，然后建立地铁站周边的车辆使用情况模型，使用数据仿真检验模型的准确性。

3.1 问题一的分析与思路

该问题需要我们解决在数据收集方式，研究区域选择和数据分析上的难点。

3.1.1 网络爬虫获取数据

因为摩拜单车的API返回的是一个正方形区域中的单车信息，只要按照一块一块的区域移动就能抓取到整个大区域的数据。我们选取北京市积水潭地铁站所在的 $1000 \times 1000(m)^2$ 正方形区域，并且每半个小时对其抓取一次数据。再对其进行去重，以消除小正方形区域的重复的部分，在输出的csv文件中，有单车的种类数量和经纬度信息等，把他们导入到googleMyMap中，标记单车点的位置，得到在积水潭地铁站附近mobike单车的分布状况。

3.1.2 数据分析的方式

原拟采用人工实地检验方式，但此方式存在大量数据误差，且耗费人力，所以转向用网络爬虫方法为主，人工检验为辅的，对手机APP及微信小程序的处理，根据确定的经纬度范围，以30分钟一次的频率收集数据。并同时截图，对比数据的真实性，同时进行了实地调查考察，一人在实地统计车辆数，另一名同学操作爬取APP上显示数据，检验GPS定位的准确性，来保证数据的真实性，有效性。

3.1.3 研究区域的选择

选择以积水潭地铁站为中心的 $1000 \times 1000(m^2)$ 范围为研究区域。我们所调查的积水潭地铁站位于2号线，属于北京地铁的高客流量地段，可以将针对该区域的研究广泛地应用到其他地区。根据北京地铁站的数据显示，平均两站中的间隔距离约为1000m左右，基于对单个地铁站的影响半

径，包含单车消费区域的足够大区域，以及数学模型的建立与应用，我们将以积水潭地铁站为中心的 $1000 \times 1000(m^2)$ 范围为研究区域。

3.2 问题二的分析与思路

整体解决北京市的单车投放数量的合理性和运营方式的问题过于庞大和复杂，因此我们把问题二的重心放在解决mobike共享单车在积水潭地铁站的“合理的单车投放数量和运营方式”，即满足消费者需求和尽早实现盈利的问题。为了便于模型的求解，采用定义1.3.2的第一种理解，最后将mobike单车积水潭地铁站的收益情况合理地放大到北京市整体，此时采用定义1.3.2的第二种理解，提出建议。

拟采用元胞自动机原理来建立数学模型。积水潭地铁站的客流量较大，因此可以认为积水潭周边的mobike单车的分布变化主要受地铁客运情况的变化；而地铁的客运是一个随时间变化的物理过程，因此建立基于元胞自动机原理的数学模型是可行的[2]。在消费者使用mobike单车时，存在多次使用的情况，所以押金收取的统计并不随着时间变化，因此我们在建立模型时不考虑押金收取情况，而是在考虑北京市整体情况时，利用实际北京地铁站数据解决“押金收取最优化”相关的问题。将模型推广应用到定义1.3.2的第二种理解上，就可以得出mobike在北京市的最优运营方案和投放数量。

四 符号说明

表 1: 符号表

符号	含义
a, b	积水潭地铁站经纬度的坐标.
x, y	该单车所在位置的经纬度坐标.
d_1, d_2	分别为东西实际距离，南北实际距离.
D	该单车所在位置距离积水潭地铁站的实际距离.
n	分组数.
s	区域内总的单车数量.
S	考虑的单车停放区域.
S_k	单车的分布和状态.
n_1	已报修的损坏的单车数.
n_2	驶出区域S的完好的单车数.
n_3	驶进区域S的单车数.
f	单车的损坏概率.
d	矩阵中的距离.
t_i	第 t_i 次单车位置变化或第 t_i 天.
L	单车距离.
α	用于支持“烧钱活动”的单日租金收入的某一固定百分比.

五 模型假设

- 设地铁站的周围地形平滑，共享单车是均匀分布的。
- 消费者选择mobike单车仅考虑距离因素，即总是选择距自己较近的单车。
- 每人仅有一次机会选择单车，选择结果仅有两种，该单车是可以使用的单车或是不可使用的单车，并进行了报修。
- 单车损坏后仍在原位置，消费者报修后马上移出研究区域。
- 无损坏的单车的驶出和单车的驶进都算作一次计费，即无损坏的车不会在一次骑车机会驶进又驶出。
- 消费者每次使用单车的时间为平均使用时间，积水潭地铁站的客流量为平均客流量[4]。

六 问题一

在此题中，我们选取较为有代表性的积水潭地铁站为调查对象，为收集数据并验证其准确性，我们采用实地调查与网络爬虫的联合方式，以积水潭地铁站为中心，选取 $1000 \times 1000(m)^2$ 的方块，用网络爬虫方式定义

- 左上角的经纬度为(39.9531318620,116.3671186765)
- 右下角的经纬度为(39.9444419839,116.3789221595)

以半个小时为间隔收取数据。（由于收取数据需要运行一定时间，所以实际时间与整点存在一定差异）

记录同一时间不同距离的数据，转换经纬度的距离为实际距离，以一系列等差数列划分区间，统计各个距离内的单车数量，反映同一时间的不同范围的单车分布；记录不同时间的范围内的单车总数，绘制折线图，反映不同时刻的单车数量的变化；利用编程绘制某时刻单车地图，对比实地调查人员的记录和手机app，体现数据可靠性。

6.1 定时刻变范围的分布分析

6.1.1 转换经纬度与实际距离

利用所查得：1经度对应实际距离的111km.1纬度对应实际距离的111km.再利用距离公式，得出计算公式为：

$$\begin{aligned}d_1 &= (x - a) \times 111(km) \\d_2 &= (y - b) \times 111 \times \cos y(km) \\D &= (d_1^2 + d_2^2)^{1/2}\end{aligned}$$

注：

- a, b : 积水潭地铁站经纬度的坐标；
- x, y : 该单车所在位置的经纬度坐标；
- d_1, d_2 : 分别为东西实际距离，南北实际距离；
- D : 该单车所在位置距离积水潭地铁站的实际距离；

6.1.2 数据分组

利用经典分组公式，计算最佳分组数，等距分组，得出公式为：

$$n = 1 + \ln s / \ln 2$$

注：

- s：区域内总的单车数量
- n：分组数

6.1.3 条形图

对条形图进行一定分析后，我们发现条形图主要有两类：一种是随着距离积水潭地铁站的越来越远，在波动中，单车数量逐渐上升。（后简称递增型）；一种是呈现类似山峰的形状，中间高两边低，随着距离积水潭地铁站的越来越远，单车数量先增后减。（后简称山峰型）

通过统计其对应的时间分布，和实地调查人员的比对，我们发现两种图形对应两种不同情形：

- 递增型：时间段为：5：00-6：30，8:00-12：40，14：53-15：56，21，30-5：00
- 山峰型：时间段为：6：30-7：30，12：40-14：53，15：56-21：30

递增型的出现时间常常为平流阶段，这段时间的人流量正常，道路交通通畅，停放量与离开量基本保持平衡，由于随着半径的逐渐增大，所统计的圆环面积逐渐增大，所以每组的单车数量在波动中逐渐变大。

山峰型则常常出现在人流量较大，道路交通拥挤的繁忙时间内，由于地铁站附近地形复杂有一个大型交叉路口，如果出现道路拥挤，则人们更偏向于提前停车后步行于地铁站，而取车的人们偏向于有车就借的心态。所以，在距离地铁站400米时会有一类类似山峰的最值，而较近较远处都会相比之下而更少一些。

下附图（图1，图2，图3），为一天中的三次主要变化：

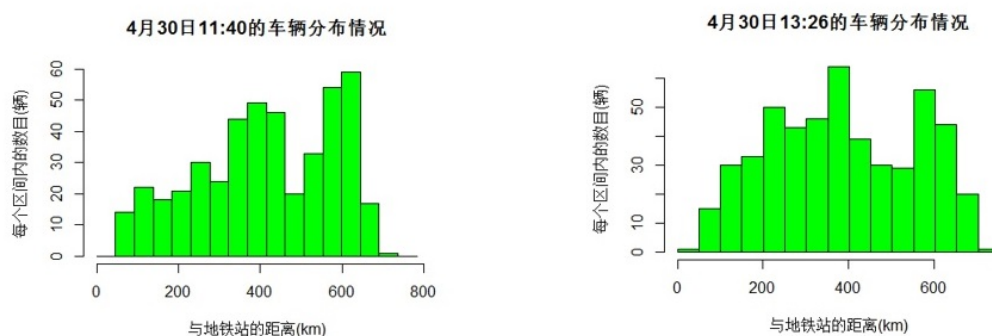


图 1: 变化I

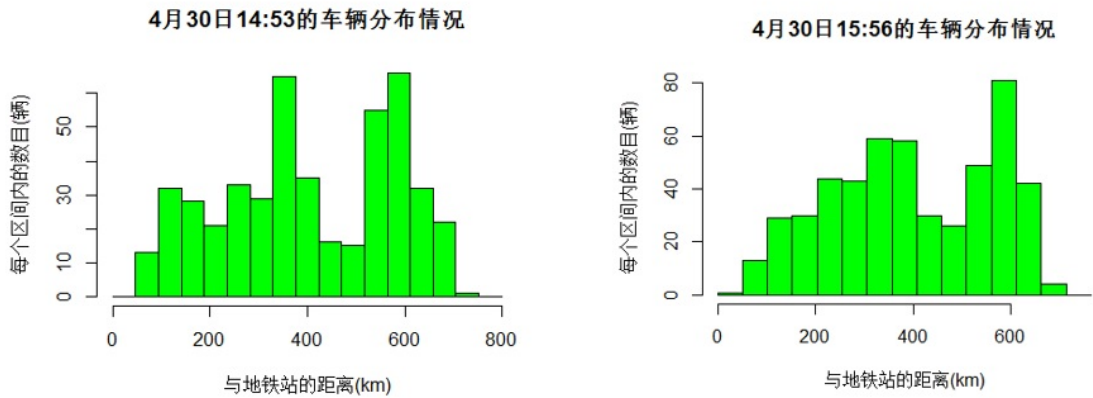


图 2: 变化II

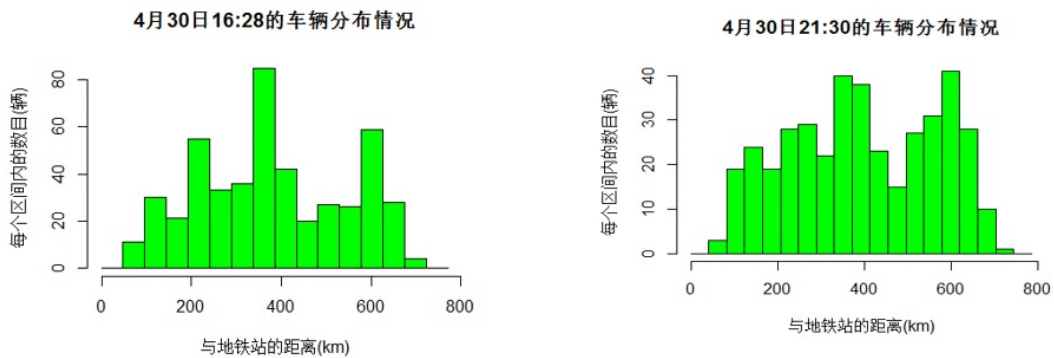


图 3: 变化III

6.2 定范围变时刻的分布分析

根据在一天内的不同时间积水潭附近正方形区域的单车数量的变化，可得到它的分布状况：

- 在早上5：00-10：00，单车数量的曲线先是一直上升到最高点，随后在9：30左右快速下降，根据实际观察，此时积水潭地铁站早上周围人流量较大，出游的人数较多，此时为人流和车流的高峰期；
- 在10：30-14：00，每两个小时就有一个较小的波动，说明此时地铁站周围1km*1km的范围内单车移动频率较小，人流密度小；
- 在15：00-17：00，单车的数量曲线在纵轴上有非常大的波动，说明此时单车移动非常频繁，此时地铁站周围的人流移动非常频繁，经过实地观察，可以发现此时人们中午下班的往返造成了这种情况；
- 在17：00-20：00时，单车数量曲线又恢复成较小的波动频率；
- 在20：00-21：30时，单车的总数量基本维持在一个水平不变，说明此时人们在地铁站周围的活动较少；
- 在21：30-23：00时，单车的数量曲线持续上升，曲线较陡峭，再继续下降，观察得知此时人们在外游玩返程途中，造成了这种变化；

- 在23: 00-24: 00时，单车的数量会先缓慢上升一个小幅度，紧接着下降一个大幅度，根据实际观察，此时人们由于各种原因由地铁站扩散到各个地方，单车的数量大幅度降低。
- 排除mobike本身API接口的问题，在24: 00-25: 00，单车的数量会逐渐稳定下来。

摩拜单车两种类型车辆的比例由数据可知为

$$T_1 : T_2 = 48 : 52$$

注：

- T_1 :第一型车辆
- T_2 :第二型车辆

具体的曲线，比例如下图（图4）所示：

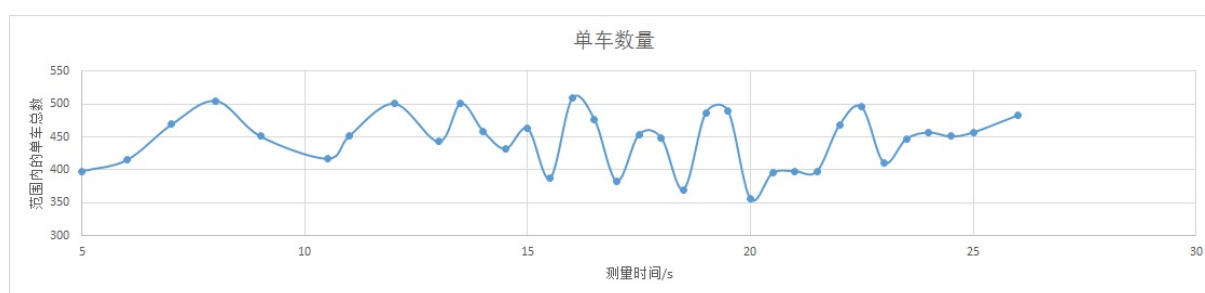


图 4: 单车的数量变化

6.3 数据可视化及合理性分析

由数据及人工实地考察比对，得出数据是准确的，可信的。当程序运行输出csv文件后，在googleMyMap中创建我的地图，添加新图层，并导入之前保存的csv 文件，得到标记有单车位置的地图，经过与实时mobike的APP上的单车的位置和数量进行比较，排除API接口带来的误差，发现两者地图上的位置等信息基本一致。地图如下图（图5）所示：

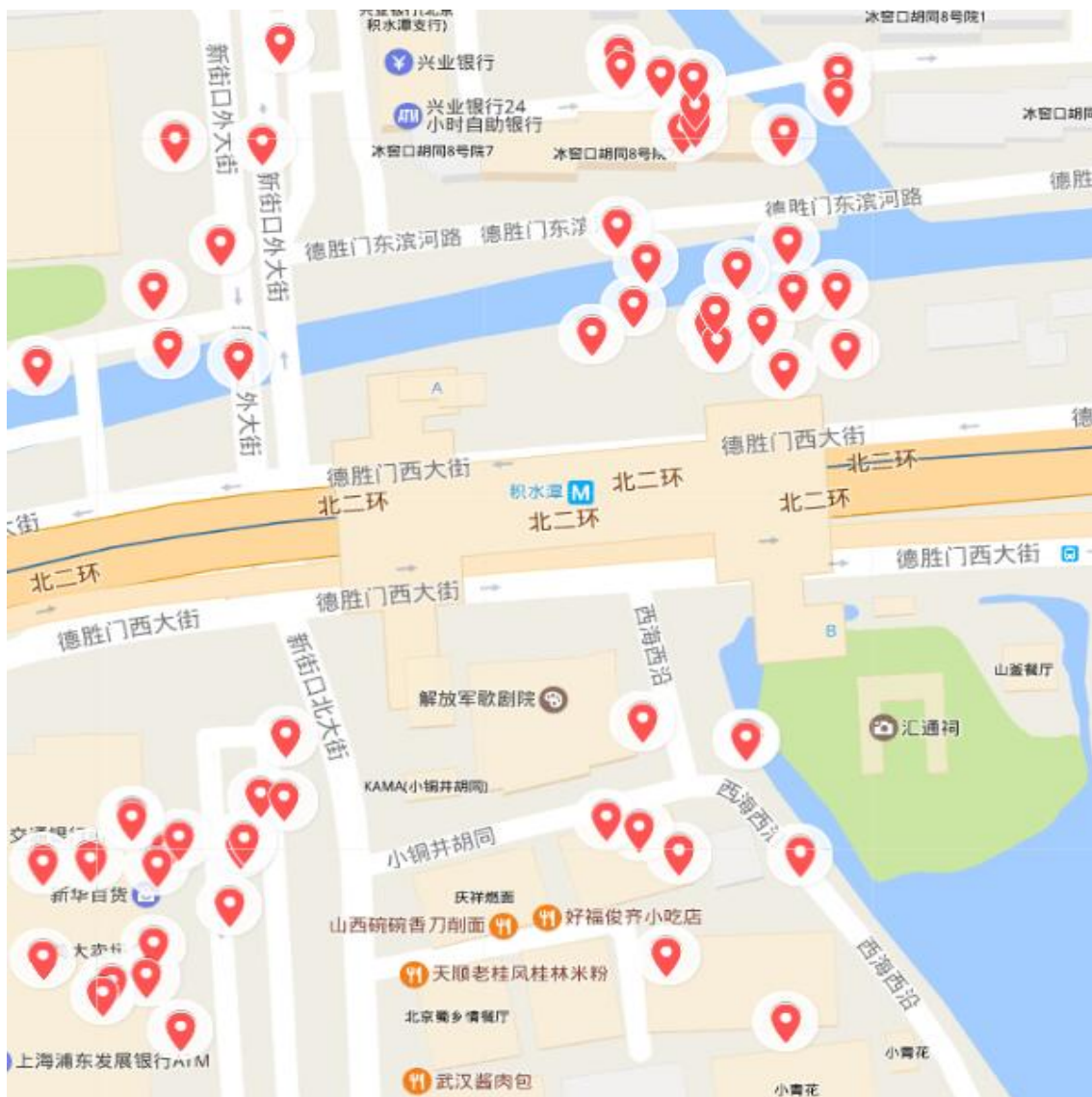


图 5: 导入谷歌地图后的结果展示

七 问题二

由于整体解决北京市的单车投放数量的合理性和运营方式的问题过于庞大和复杂，因此我们把问题二的重心放在解决mobike共享单车在积水潭地铁站的“合理的单车投放数量和运营方式”，即满足消费者需求和尽早实现盈利的问题，为便于计算求解模型时，仅考虑定义2 的第一种情况。最后将模型合理地推广应用到北京市整体，根据模型给出运营优化方案。考虑最佳运营方式在定义2的第二种情况下，是否成立，并提出建议。

定义 1 满足消费者需求：在一段时间内，在某区域的单车可利用数量始终大于等于同一时刻的该区域单车消费者的数量。

定义 2 尽早实现盈利：在规定的完整时间段内，在某区域中单车依靠押金利息和租金收入获得的总体收入大于其损毁维护和活动优惠带来的总体支出，而且总体利润不小于同一条件下其他共享单车的平均总体利润。或考虑共享单车的制造成本，mobike品牌领先于其他同行进入盈利状态。

定义 3 损坏及报修：单车的轻度损坏和重度损坏。由于重度损毁的单车位置无法在地图上显示，

因此在模型中的单车损坏情况均为轻度损坏，平均每辆每次损失算作100元[3]。车辆损坏后仍在原位置，消费者报修后马上移出研究区域 S 。

7.1 模型一

积水潭周边的mobike单车分布情况主要由地铁站的客运情况决定，考虑在积水潭地铁站周边的单车投放量，租金，活动优惠和车辆损毁因素，仅考虑定义2 的第一种理解，忽略单车初始分布及状态情况的影响，仅考虑单车的分布及状态的变化规律，建立模型一。

7.1.1 模型假设

- 将研究时间定为两周,活动定为每周特定一天，即第1天和第7天献出租金收入的20%
- 一小时内共享单车的消费者人数服从正态分布

7.1.2 流程图

首先，给出单车的分布及状态矩阵 S_k ,将已报修车辆数 n_1 ，离开车辆数 n_2 以及停靠车辆数 n_3 的值统一为0，其次判断单车是否发生损坏，并对单车状态进行更新。如果有消费者选择该车，则判断该车是否损坏，如损坏则立马将该车进行报修，该位置变为空位，参数 n_1 自加，然后判断位置变化时期是否为优惠期，若是，则数量变化为原数量的80%；否则，不做改变；如未损坏，则该位置同样为空， n_2 自加。如有单车驶进区域 S 的空位，然后 n_3 自加，该位置为非空。判断单车的状态及分布变化的次数是否满足其要求，若是，则结束流程，反之再次判断车辆的损坏情况，继续循环。如图（图6）所示：

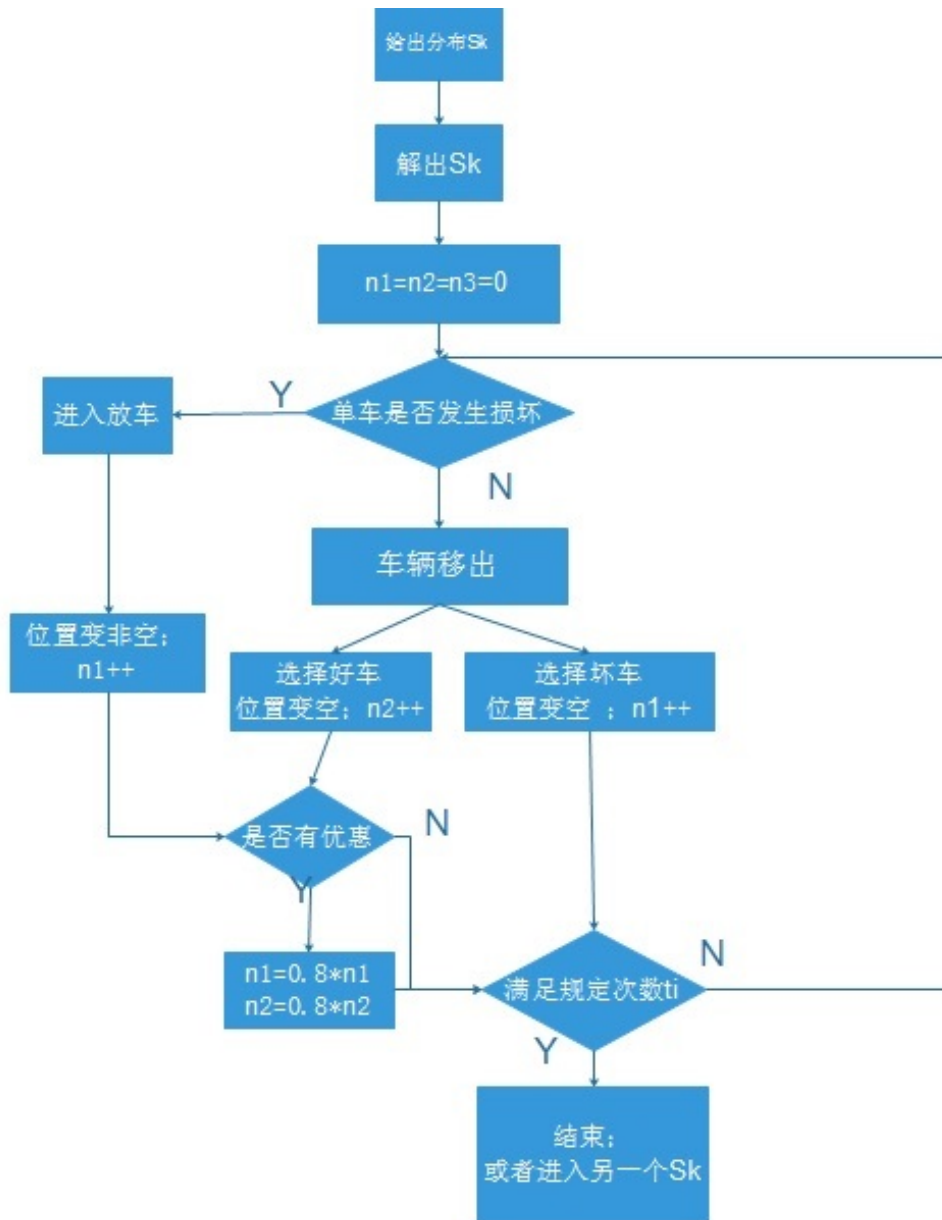


图 6: 模型流程图

7.1.3 消费者数量确定

据北京地铁官网资料(表2)显示, 积水潭地铁站运营时间为5:30——22:30。一天内的单位小时的最大人流数为当天总人数的0.13, 由最新资料显示, 2号线一天的客流量为112.13万次, 2号线共有13站, 假设站间客流量不存在差异, 则一天内经过积水潭的客流量为8.63万次, 按比例则可得出一小时最大客流量为1.122万人数次, 同理计算标准差为0.035万人次。查阅资料[4]显示, 使用共享单车的人数占总出行人数的额11.6%。综上所述, 假设共享单车的消费者人数服从正态分布, 则积水潭地铁站一小时内最大使用共享单车的消费者均值为1300人, 标准差为350人。

表 2: 地铁站数据表	
时间段	人流量
5 : 00 – 6 : 00	49000.
6 : 00 – 7 : 00	190000.
7 : 00 – 8 : 00	485000.
8 : 00 – 9 : 00	449000.
9 : 00 – 10 : 00	215000.
10 : 00 – 11 : 00	152000.
11 : 00 – 12 : 00	135000.
12 : 00 – 13 : 00	145000.
13 : 00 – 14 : 00	152000.
14 : 00 – 15 : 00	152000.
15 : 00 – 16 : 00	162000.
16 : 00 – 17 : 00	200000.
17 : 00 – 18 : 00	385000.
18 : 00 – 19 : 00	390000.
19 : 00 – 20 : 00	185000.
20 : 00 – 21 : 00	125000.
21 : 00 – 22 : 00	105000.
22 : 00 – 23 : 00	50000.

7.1.4 问题各参数矩阵化

mobike单车的长度经实际测量为1.67m，宽度约占0.5m，考虑到允许行人通过的面积，确定单车停靠面积为 $2m \times 1m$ 。由于区域 S 的面积确定为 $1000m \times 1000m$ ，并且考虑到Matlab的计算能力，将单车停靠位置转化为矩阵，并令矩阵大小为 50×100 。矩阵中仅含“0”，“1”，“2”三种元素，表示单车的停靠状态，“0”表示该位置为空位，没有单车停靠或是不可停靠区域；“1”表示该位置有单车停靠，且单车没有损坏；“2”表示该位置有已损坏的单车停靠。按照此规则将各个参数区域均实行矩阵化，仍保留原符号表示。

7.1.5 模型的实现与检验

基于元胞自动机原理，编写数学模型并实现的部分结果如下：

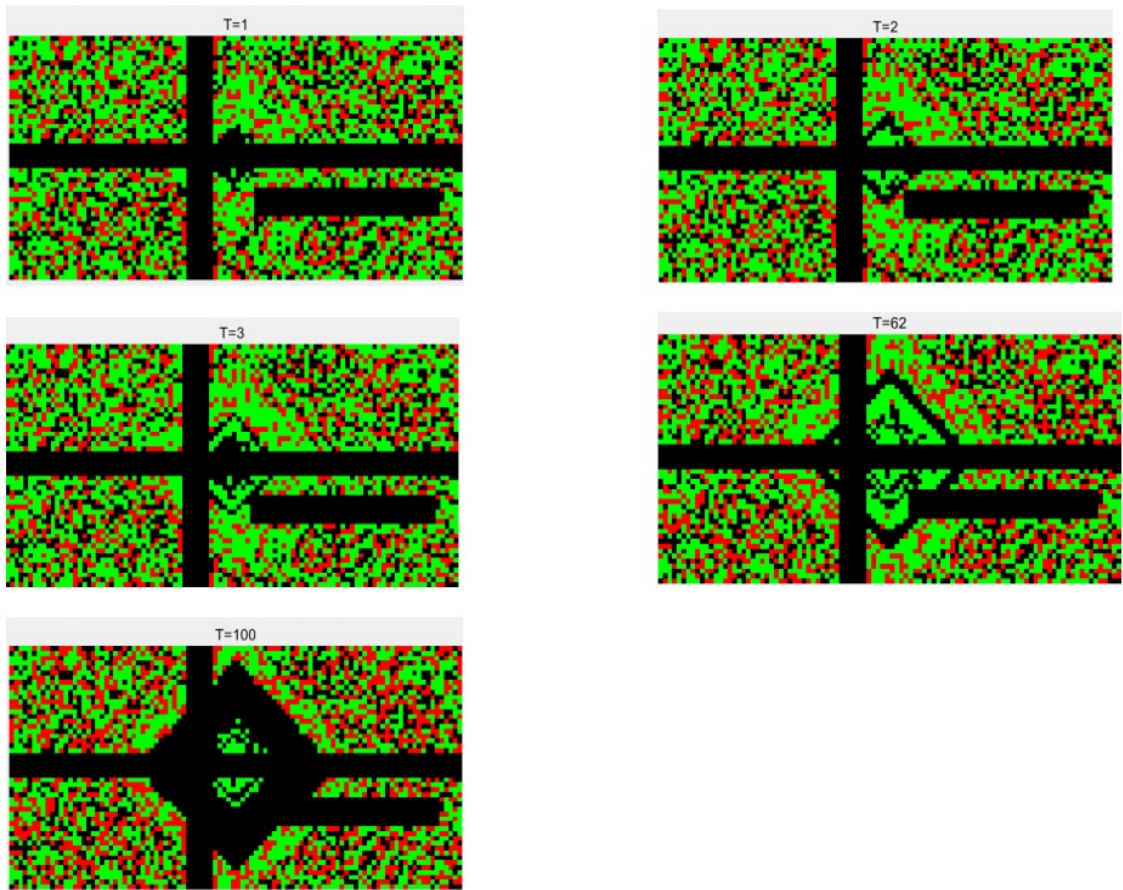


图 7: 模型一结果部分展示

其中 t_i 表示第 t_i 次的单车位置情况变化，红色表示“2”，绿色表示“1”，黑色表示“0”。

根据模型运行结果可知，在任意给定的关于单车分布及状态的先决条件下，单车的分布及状态变化规律基本符合实际情况，因此认为模型一是比较合理的，并以模型一为基础建立模型二。

7.2 模型二

定义 4 大小为 50×100 的矩阵的矩阵中心为

$$O = (25.5, 50.5)$$

简称中心 O ，或者 O 。

定义 5 矩阵中的距离：对于矩阵中的任意两个元素的位置 $(x_1, y_1), (x_2, y_2)$ ，这两个元素在矩阵中的距离

$$d = |x_1 - x_2| + |y_1 - y_2|$$

简称距离，记为 d 。(在欧式平面中，两点 $(x_1, y_1), (x_2, y_2)$ 之间的距离 d 可以定义为 $d = |x_1 - x_2| + |y_1 - y_2|$ ，它与 $(x_1 - x_2)^2 + (y_1 - y_2)^2$ 的平方根是等价的[5]。同样地，我们可以定义矩阵中元素位置 (i, j) 到矩阵中心 O 的距离 d = 在矩阵中定义一个单车距离为)

定义 6 单车距离：矩阵中相邻元素之间在矩阵中的距离对应的实际距离，记为 L 。

建立模型二是为了解决积水潭地铁站周边区域 S 的单车最佳投放数量。最佳投放数量是指满足定义1，定义2的数量。根据假设，最佳的分布方式是指优先选择与 O 距离最短的位置进行投放。

7.2.1 模型优化

为了简化运算，同时保证模型的科学性，做出如下变动，

- 初始摆放时，单车不存在损坏情况，即在矩阵中全为“1”。
- 将 t_i 变更表示为研究的第 t_i 天。并针对此变化，将模型中单车状态的变化的次数进行调整。
- 由于主要考虑单车的投放数量，所以每班地铁中选择单车的人数定为65，而不是选则以65 为中心的正态数据。
- 由于客运量固定，所以进入区域 S 的单车数量和驶出 S 的单车数量都是不变的。

因此，将 t_i 确定为1，首先在矩阵 S 内选取满足最佳分布方式的与 O 的距离小于等于 r_i 的元素位置。 $(r_i = 1, 2, 3 \dots, 25, i = 1, 2, 3 \dots, 25)$ 根据收益选取最佳半径 r_i ，再将半径 r_i 和中心 O 决定的区域，进一步细化，确定出单车的最佳投放数量。

7.2.2 运行结果

由于区域 S 存在不可放置的位置，而且单车的初始摆放数量必须大于等于65，可以得出 r_i 的范围为11至25。

这里的 M 是一个1行15列的矩阵，给出的是在第一天内的关于半径 r_i 从 $11L - 25L$ 的盈利。

可以看出在距中心 $14L$ 时，既能够满足消费者的需求，也能够获得最大的收益。因此在接下来的研究中，我们针对这一种分布 $L=14$ 时进行研究。以一种情况为例，结果如下图8所示：

因为投放之初，已做假设所有的车辆均无损坏，所以图9中只会出现两种情况，就是空车位或不可停靠，以及有好车的情况，所以图中只有绿色和黑色。根据该种最佳投放方式，建立模型三。

```

M =

    1.0e+03 *

Columns 1 through 6

    1.9516    2.0532    2.0532    2.1549    1.9516    1.7482

Columns 7 through 12

    2.0532    2.1549    2.0532    1.8499    2.1549    1.9516

Columns 13 through 15

    2.1549    1.8499    1.8499

```

图 8: 运行结果展示

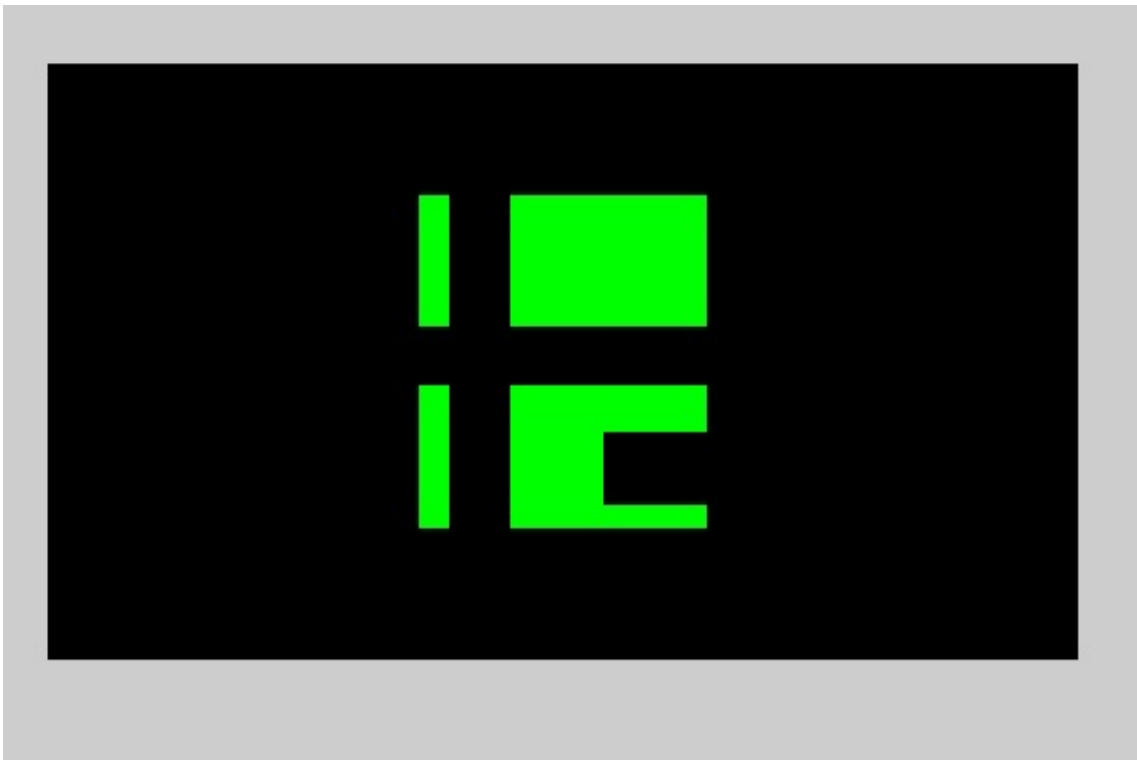


图 9: 运行结果展示

7.2.3 模型三

解决在最佳投放方式和最佳分布方式的情况下，优惠活动对总收益的影响

7.2.4 模型假设

- 将优惠活动视为把单日内单车的租金收入的某一固定百分比 α 用于支持“烧钱活动”，不产生盈利，会造成次日消费者使用量增加 $\alpha/10$

- 将 t_i 变更表示为研究的第 t_i 天。并针对此变化，将模型中单车状态的变化的次数进行调整。

表 3: 地铁站数据表	
用于活动的百分比 α	利润money
0	1.1906e+06.
0.1	9.6761e+05.
0.2	9.1789e+05.
0.3	7.6441e+05.
0.4	6.4634e+05.

对模型数据进行处理后，左边一列数据表示“烧钱活动”所消耗的收益占原收益的比例，右边一列数据表示mobike单车两周内在积水潭地铁站周边的总收益。

根据模型三运行得到的数据，随着“烧钱活动”占单日总收益的比例增大，mobike单车在两周内总的收益是减小的。所以可以得出结论:mobike单车的“烧钱活动”并不能在短期内取得消费者数量的大量增长以及收益的增加，反而导致收益的下降；为了在短期内尽早实现盈利，应该取消“烧钱活动”。

7.2.5 在北京市层面的推广

由模型二我们得出了积水潭地铁站的最佳投放方式为投放446辆，以靠近中心 O 为优先放置单车。由模型三我们得出在发红包的使用率增高与利润的直接损失之间，还是没有红包时利润最大。

由此推广到北京市的范围内，据北京地铁官网显示，北京一共有大小地铁站198个2个暂停，由于繁华区域地铁站多，人流少的区域地铁站少，所以均衡来看可以把积水潭看作平均水平估计北京市对共享单车的需求量。所以可以求得，在最开始的投放量约需要9万辆左右。这仅仅是地铁站附近的车辆使用状况，所以对共享单车的最终投放量只能起到一定的建议；但是，在红包活动中的结论，则具有指导意义，分析后数据明确显示了红包活动会大幅降低利润，虽然会激励用户增加使用，减少调度费用，但若想短期内获利，建议短期终止此种“烧钱行为”。如按此方式运营，根据日前公布的数据显示北京的注册用户约有3000万，两周内的运营利润约为34.2万元。

八 模型的评价

8.1 模型优点

针对问题一，利用网络爬虫得到足够多的数据，便于分析；采用多角度分析，图文结合的方式，使数据的规律表现得形象具体，十分详细阐述了共享单车的分布和使用规律。

针对问题二，采用的元胞自动机模型可以通过对研究区域内单个位置上的单车变化情况的模拟，可以求出单车的租金收益和维修成本，使得模型对现实有很好的解释和预测作用。

8.2 模型缺点

构建的元胞自动机模型对消费者的选车行为方式有严格的限制，与实际情况有较大差异。因此对模型计算出的结果的精确性有影响。

解决问题二时在考虑客流量时并没有严格考虑时间对其的影响，而在现实生活中时间对客流量有较大影响。因此对模型算出的结果有所影响。

参考文献

- [1] 李敏莲, 共享单车市场调研与分析
- [2] 张宏军, 物理系统的元胞自动机模拟
- [3] <http://www.huoche.net/show_506096/>
- [4] <<http://www.cctime.com/html/2017-4-26/1283101.htm>>
- [5] 邓冠铁, 复变函数论

九 给mobike的一封信

您好，mobike的CEO：

我们是一组使用mobike共享单车的消费者，在对积水潭地铁站附近的mobike单车的使用情况进行调查后，我们对于mobike单车目前在北京的运营有了较为深刻的了解，所以在做了一系列调查后，我们对单车投放量和运营方式给出以下几点建议，希望您能采纳：

- 对于mobike在北京的单车投放量

经过调查研究，单车使用率最高的地方一般毗邻商圈等繁华地区，地铁站等交通要道，所以有地铁站的周围的单车使用情况来推理北京市的大部分地区情况，是合理的。在对积水潭调查时，我们发现在最初投入时，以446辆单车的初始投入量比较适宜，且最好紧邻地铁站投入，增加单车使用率。并且对于报修车辆建议及时移除送修，有利于地铁站周边的单车使用。根据目前北京有200余站地铁站，目前贵公司已在北京投入车辆30万辆，所以，我们建议减少无效投入，增强有效发展，即增强在地铁站等交通要道的周围的单车数量，所以我们建议再次投入时可针对地铁站投入约10万的单车。

- 对于mobike在北京的运营方式

目前据我们了解贵公司的活动主要有“红包车”“微信小程序优惠”等，虽然这些方式看似可以节省部分人工调遣的劳务费，但在目前短期内很难获利，且常常属于亏损状态，以我们所模拟的数周利润来看，此种优惠活动始终对利润的提升起着不良作用，就以我们建议的方式投入且不添加活动仍需667天才可以真正挽回成本，所以建议在短期内减少活动频率，以积累资本，在公司盈利后再进行扩展市场，增大市场规模，回馈客户。由于调查中也有反应押金退还不及时，与中信银行合作不透明导致的信任度的降低。也要及时处理。

- 对于mobike的APP以及小程序的应用

无论是mobike的APP还是微信小程序，它们的API接口都存在局限性，因为它们都只能显示当前GPS定位所在的一个小正方形区域内的单车的位置，其数量大部分都限制在24-29辆，而且GPS轨迹显示短时间内单车一直在附近跳动，显然这是不合理的，这就造成了有时候看到车但是其实那里没有车的情况。不利于用户体验，很可能加剧消费者的反感程度，所以建议贵公司可以更加注重细节的开发，解决此类问题。

希望您能早点让mobike的开发团队解决这些问题，让mobike带给我们更多便利和快乐。

此致

敬礼

Mobike的一组用户

附 录

0.1 问题一:数据分组以及作图

```
'''{r}
B<-read.table("C:/Users/ASUS1022/Desktop/数据/2256.csv",header=T,sep=",")
a<-B$x
b<-B$y
p=116.3726708
q=39.94872755
A<-(a-p)*111
C<-(b-q)*111*cos(b)
K<-sqrt(A^2+C^2)
k<-1000*round(K,2)
s<-length(k)
n<-1+log(s)/log(2)
n
##x<-rnorm(1000,3,4)
brs<-seq(0,800,s/n)
D1<-table(cut(k,br=brs))
D1
hist(k,breaks=brs,freq = T,col="green",density = NULL,main = paste
("4月30日22:56的车辆分布情况"),xlab = "与地铁站的距离(km)",ylab= "每个区间内的数
目(辆)")
'''
```

0.2 问题一:网络爬虫及地图导出

```
import datetime
import os
import os.path
import random
import sqlite3
import threading
import time
import ujson
from concurrent.futures import ThreadPoolExecutor

import numpy as np
import requests
from retrying import retry

from modules.ProxyProvider import ProxyProvider
```

```

class Crawler:
    def __init__(self):
        self.start_time = datetime.datetime.now()
        self.csv_path = "./db/" + datetime.datetime.now().strftime("%Y%m%d")
        os.makedirs(self.csv_path, exist_ok=True)
        self.csv_name = self.csv_path + "/"
        + datetime.datetime.now().strftime("%Y%m%d-%H%M%S") + '.csv'
        self.db_name = "./temp.db"
        self.lock = threading.Lock()
        self.proxyProvider = ProxyProvider()
        self.total = 0
        self.done = 0

    def get_nearby_bikes(self, args):
        try:
            url = "https://mw.xmobike.com/mobike-api/rent/nearbyBikesInfo.do"

            payload = "latitude=%s&longitude=%s&errMsg=getMapCenterLocation" % (args[0], args[1])

            headers = {
                'charset': "utf-8",
                'platform': "4",
                "referer": "https://servicewechat.com/wx40f112341ae33edb/1/",
                #'referer': "https://servicewechat.com/",
                'content-type': "application/x-www-form-urlencoded",
                #'user-agent': "MicroMessenger/6.5.4.1000 NetType/WIFI Language/zh_CN",
                #'host': "mw.xmobike.com",
                'connection': "Keep-Alive",
                'accept-encoding': "gzip",
                'cache-control': "no-cache"
            }

            self.request(headers, payload, args, url)
        except Exception as ex:
            print(ex)

    def request(self, headers, payload, args, url):
        while True:
            proxy = self.proxyProvider.pick()
            try:
                response = requests.request(
                    "POST", url, data=payload, headers=headers,

```

```

        proxies={"https": proxy.url},
        timeout=5,verify=False
    )

    with self.lock:
        with sqlite3.connect(self.db_name) as c:
            try:
print(response.text)
                decoded = ujson.decode(response.text)['object']
                self.done += 1
                for x in decoded:
                    c.execute("INSERT INTO mobike VALUES (%d,'%s',%d,%d,%s,%s,%f,%f)" % (
                        int(time.time()) * 1000, x['bikeIds'], int(x['biketype']), int(x['distId']),
                        x['distNum'], x['type'], x['distX'],
                        x['distY']))

                timespend = datetime.datetime.now() - self.start_time
                percent = self.done / self.total
                total = timespend / percent
                print(args, self.done, percent * 100, self.done / timespend.total_seconds() * 60, total,
                    total - timespend)
            except Exception as ex:
                print(ex)
                break
        except Exception as ex:
            proxy.fatal_error()

def start(self):

    left = 39.9531318620
    top = 116.3671186765
    right = 39.9444419839
    bottom = 116.3789221595

    offset = 0.002

    if os.path.isfile(self.db_name):
        os.remove(self.db_name)

    try:

```

```

        with sqlite3.connect(self.db_name) as c:
            c.execute('''CREATE TABLE mobike
(Time DATETIME, bikeIds VARCHAR(12), bikeType TINYINT,distId INTEGER,distNum TINYINT,
type TINYINT, x DOUBLE, y DOUBLE)''')
except Exception as ex:
    pass

    executor = ThreadPoolExecutor(max_workers=250)
    print("Start")
    self.total = 0
    lat_range = np.arange(left, right, -offset)
    for lat in lat_range:
        lon_range = np.arange(top, bottom, offset)
        for lon in lon_range:
            self.total += 1
            executor.submit(self.get_nearby_bikes, (lat, lon))

    executor.shutdown()
    self.group_data()

def group_data(self):
    print("Creating group data")
    conn = sqlite3.connect(self.db_name)
    cursor = conn.cursor()

    f = open(self.csv_name, "w")
    for row in cursor.execute('''SELECT * FROM mobike'''):
        timestamp, bikeIds, bikeType, distId, distNumber, type, lon, lat = row
        f.write("%s,%s,%s,%s,%s,%s,%s,%s\n" % (
            datetime.datetime.fromtimestamp(int(timestamp) / 1000).isoformat(),
            bikeIds, bikeType, distId, distNumber, type, lon, lat))
    f.flush()
    f.close()

    os.system("gzip -9 " + self.csv_name)

Crawler().start()

```

0.3 问题二

0.3.1 一次分布变化的租金和成本

```
function u=prof(a,b,c)
u=a*(-100)+2.072*(b+c);
```

0.3.2 就近原则

```
function u=turn1(A)
t11=sum(A,1);t12=sum(t11,2);
a=randn;
b=round(t12);c=round(65);
b=min(b,c);
U=zeros(3,t12);
k0=1;j0=1;
    for i=1:1:size(A,1);
        for k=k0:1:t12;
            for j=j0:1:size(A,2);
                if A(i,j)==1;
                    U(:,k)=[i,j,(abs(i-25.5)+abs(j-50.5))];
                    k0=k+1;j0=j+1;break;
                end;
            end;
        end;
        j0=1;
    end;
    for p=1:1:(t12-1);
        for q=(p+1):1:t12;
            if U(3,p)>U(3,q);
                e=U(:,p);U(:,p)=U(:,q);U(:,q)=e;
            end;
        end;
    end;
    for v=1:1:b;
        A(U(1,v),U(2,v))=0;
    end;
u=A;
```

0.3.3 模型一初始随机分布

```
close all;clc;clear;
figure;
f=0.001;%破坏概率
```

```

axes;rand('state',0);
set(gcf,'DoubleBuffer','on');
S=round(rand(50,100)*2);
Sk=zeros(50,100);
Sk(1:50,1:100)=S;
%建立空白区域
Sk(23:27,:)=0;
Sk(:,40:45)=0;
Sk(32:37,55:95)=0;
n1=0;n2=0;n3=0;
%红色表示车辆损坏，S中等于2的位置
%绿色表示车辆完好，S中等于1的位置
%黑色表示空格位置，S中等于0的位置
C=zeros(50,100,3);
R=zeros(50,100);
G=zeros(50,100);
R(Sk==2)=1;G(Sk==1)=1;
C(1:50,1:100,1)=R;
C(1:50,1:100,2)=G;
Ci=imshow(C);ti=0;
tp=title(['T=',num2str(ti)]);
while 1;
    ti=ti+1;
    St=Sk;
    %车辆损坏
    Sf=zeros(50,100);
    Sf(Sk==1)=1;
    St=St+Sf.*(rand(50,100)<f);
    %车辆运出
    St1=zeros(50,100);
    St2=zeros(50,100);
    St1(St==2)=1.5;
    St2(St==1)=1;
    Stin=St1+St2;
    Stin1=floor(Stin);
    Stin1=turn1(Stin1);
    S1=Stin-Stin1;
    n1=length(find(S1==1.5))+n1;%运出坏车数目
    if (ti<340)|| (ti>2379&&ti<2720);
        n2=0.8*length(find(S1==1))+n2;
    else n2=length(find(S1==1))+n2;
    end;%骑出好车数量
    S2=zeros(50,100);

```

```

S2(S1==1.5)=0;S2(S1==0.5)=2;S2(S1==0)=1;
S2(S1==1)=0;S2(S1==0&Stin==0)=0;%变换原则
S2(23:27,:)=0; %建立空白区域
S2(:,40:45)=0;
S2(32:37,55:95)=0;
%好车骑进
St3=zeros(50,100);
St3(St==0)=1;
St3(23:27,:)=0; %建立空白区域
St3(:,40:45)=0;
St3(32:37,55:95)=0;
St3=St3-turn1(St3);
    if (ti<340)|| (ti>2379&&ti<2720);
        n3=0.8*sum(sum(St3,1),2)+n3;
    else n3=sum(sum(St3,1),2)+n3;
    end;%骑进的车辆数
St=S2+St3;
%车辆损坏
Sg=zeros(50,100);
Sg(St==1)=1;
St=St+Sg.*(rand(50,100)<f);
Sk=St;
R=zeros(50,100);
G=zeros(50,100);
R(Sk==2)=1;
G(Sk==1)=1;
C(:, :, 1)=R;
C(:, :, 2)=G;
set(Ci, 'CData', C);
set(tp, 'string', ['T=', num2str(ti)]);
pause(3);
end

```

0.3.4 最佳投放数量及方式

```

f=0.001;%破坏概率
figure;axes;
a=cir;
set(gcf, 'DoubleBuffer', 'on');

```

```

S=a(:, :, 14);
Sk=zeros(50,100);
Sk(1:50,1:100)=S;
%建立空白区域
Sk(23:27, :)=0;
Sk(:, 40:45)=0;
Sk(32:37, 55:95)=0;
C=zeros(50,100,3);
R=zeros(50,100);
G=zeros(50,100);
R(S==2)=1;G(S==1)=1;
C(1:50,1:100,1)=R;
C(1:50,1:100,2)=G;
Ci=imshow(C);
n=sum(sum(Sk,1),2);

```

0.3.5 最佳投放数量及方式

```

function u=cir
%画出区域
S=zeros(50,100);S1=zeros(50,100);S2=zeros(50,100);S3=zeros(50,100);
S1(23:27, :)=1;S2(:, 40:45)=1;S3(32:37, 55:95)=1;
A=zeros(50,100,25);
%建立圆墙
for i=1:1:25;
    S((26-i):(25+i), (51-i):(50+i))=1;
    S((S==1)&(S1==1))==0;S((S==1)&(S2==1))==0;S((S==1)&(S3==1))==0;
    A(:, :, i)=S;
    S=zeros(50,100);
end;
u=A;

```

0.3.6 最佳方案的盈利（不考虑成本）

```

close all;clc;clear;
f=0.001;%破坏概率
a=cir;
S=a(:, :, 14);
Sk=zeros(50,100);

```

```

Sk(1:50,1:100)=S;
%建立空白区域
Sk(23:27,:)=0;
Sk(:,40:45)=0;
Sk(32:37,55:95)=0;
n1=0;n2=0;n3=0;money=prof(n1,n2,n3);
%红色表示车辆损坏，S中等于2的位置
%绿色表示车辆完好，S中等于1的位置
%黑色表示空格位置，S中等于0的位置
ti=0;
while ti<13;
    ti=ti+1;
    St=Sk;
    %车辆损坏
    Sf=zeros(50,100);
    Sf(Sk==1)=1;
    St=St+Sf.*(rand(50,100)<f);
    %车辆运出
    St1=zeros(50,100);
    St2=zeros(50,100);
    St1(St==2)=1.5;
    St2(St==1)=1;
    Stin=St1+St2;
    Stin1=floor(Stin);
    Stin1=turn1(Stin1);
    S1=Stin-Stin1;
    n1=length(find(S1==1.5))+n1;%运出坏车数目
    n2=length(find(S1==1))+n2;
    %骑出好车数量
    S2=zeros(50,100);
    S2(S1==1.5)=0;S2(S1==0.5)=2;S2(S1==0)=1;
    S2(S1==1)=0;S2(S1==0&Stin==0)=0;%变换原则
    S2(23:27,:)=0;%建立空白区域
    S2(:,40:45)=0;
    S2(32:37,55:95)=0;
    %好车骑进
    St3=zeros(50,100);
    St3(St==0)=1;
    St3(23:27,:)=0;%建立空白区域
    St3(:,40:45)=0;
    St3(32:37,55:95)=0;
    St3=St3-turn1(St3);
    n3=sum(sum(St3,1),2)+n3;%骑进的车辆数

```

```

St=S2+St3;
%车辆损坏
Sg=zeros(50,100);
Sg(St==1)=1;
St=St+Sg.*(rand(50,100)<f);
Sk=St;
money=max(money,340*prof(n1,n2,n3));
end;
money

```